

Таблицы и ключи

Вымышленный мини-курс «Полка» объясняет устройство реляционной базы данных на примере каталога учебных материалов. Все названия и числовые примеры искусственные.

- 1.1 База данных - организованная совокупность данных. Система управления базами данных хранит эти данные и управляет доступом к ним.
- 1.2 В реляционной модели данные представлены таблицами. Строка таблицы описывает отдельную запись, а столбец - один из её атрибутов.
- 1.3 Схема таблицы задаёт названия столбцов, типы данных и ограничения. Тип данных определяет допустимый вид значений столбца.
- 1.4 Первичный ключ однозначно определяет строку. Значения первичного ключа уникальны, а входящие в него столбцы не могут содержать NULL.
- 1.5 Первичный ключ может состоять из одного столбца или из нескольких. Ключ из нескольких столбцов называется составным.
- 1.6 Суррогатный ключ создаётся специально для идентификации записи и не несёт предметного смысла. В учебном каталоге таким ключом служит числовой id.
- 1.7 NULL обозначает отсутствие известного значения. NULL не равен числу ноль и не является пустой строкой.
- 1.8 Ограничение NOT NULL запрещает отсутствие значения. Ограничение CHECK проверяет логическое условие; например, размер файла должен быть больше нуля.

ПРОВЕРОЧНЫЙ ПРИМЕР

В таблице subjects есть запись id=7, title=Базы данных. В таблице materials есть запись id=101, title=Таблицы, size_bytes=2048. Числа 7 и 101 являются искусственными идентификаторами, а 2048 - размером учебного файла в байтах.

Связи и нормализация

Связи помогают хранить связанные записи без неоднозначности. В этом разделе одна таблица описывает предметы, другая - материалы, а третья - теги.

- 2.1** Связь «один ко многим» означает, что одной записи первой таблицы может соответствовать несколько записей второй. В каталоге у одного предмета может быть много материалов.
- 2.2** Внешний ключ связывает таблицы. В нашем примере `materials.subject_id` ссылается на первичный ключ `subjects.id`.
- 2.3** Ссылочная целостность требует, чтобы непустое значение внешнего ключа указывало на существующую запись. Материал с `subject_id=7` требует существования предмета с `id=7`.
- 2.4** Связь «многие ко многим» реализуют через промежуточную таблицу. В каталоге один материал может иметь много тегов, и один тег может относиться к нескольким материалам.
- 2.5** Промежуточная таблица `material_tags` содержит `material_id` и `tag_id`. Составной первичный ключ из этих двух столбцов запрещает повтор одной и той же пары.
- 2.6** Для первой нормальной формы в принятой учебной модели каждая ячейка содержит одно значение. Список тегов через запятую не хранится вместо отдельных связанных записей.
- 2.7** Избыточное повторение сведений создаёт риск несогласованных изменений. Поэтому название предмета хранится в `subjects`, а материалы ссылаются на его `id`.
- 2.8** Правило `ON DELETE CASCADE` удаляет зависимые строки вместе с родительской. В учебном примере удаление предмета удаляет его материалы, если для этой связи явно задано такое правило.

ПРОВЕРОЧНЫЙ ПРИМЕР

Материалы 101 и 102 относятся к предмету 7. Тег 5 связан с обоими материалами: в `material_tags` записаны пары (101, 5) и (102, 5). Вторая запись пары (101, 5) нарушит составной первичный ключ.

Запросы и индексы

Запрос на чтение выбирает нужные записи и вычисляет результат. Индекс помогает находить данные, но не заменяет ограничения целостности или осмысленный запрос.

- 3.1** Операция SELECT читает данные. Само чтение каталога не добавляет новые материалы и не изменяет сохранённые названия.
- 3.2** Условие WHERE отбирает строки по заданному признаку. Например, условие `subject_id=7` выбирает материалы только этого предмета.
- 3.3** ORDER BY задаёт порядок строк результата. Без явной сортировки приложение не должно полагаться на порядок, в котором база вернула строки.
- 3.4** Операция JOIN объединяет данные связанных таблиц по условию. Соединение `materials.subject_id` с `subjects.id` позволяет получить название предмета вместе с материалом.
- 3.5** COUNT(*) считает все строки выбранного набора. COUNT(column) считает только строки, в которых указанный столбец не содержит NULL.
- 3.6** GROUP BY объединяет строки в группы для вычисления агрегатов. Группировка материалов по `subject_id` позволяет отдельно посчитать материалы каждого предмета.
- 3.7** Индекс - дополнительная структура данных для поиска. Он может ускорить подходящие запросы, но занимает место и требует обновления при изменении индексируемых данных.
- 3.8** Обычный индекс сам по себе не запрещает одинаковые значения. Уникальный индекс дополнительно контролирует уникальность согласно правилам выбранной СУБД.

ПРОВЕРОЧНЫЙ ПРИМЕР

Пусть у предмета 7 есть два материала, а у предмета 8 - один. Группировка по `subject_id` с подсчётом строк даст (7, 2) и (8, 1). Чтобы показать результат по возрастанию номера предмета, нужно явно задать сортировку.

Транзакции и восстановление

Транзакция объединяет связанные изменения в одну логическую операцию. Сохранность данных проверяют не только успешной записью, но и возможностью восстановления.

- 4.1 COMMIT подтверждает изменения транзакции. ROLLBACK отменяет её ещё не зафиксированные изменения.
- 4.2 Атомарность означает принцип «всё или ничего»: связанные изменения одной транзакции фиксируются вместе либо не фиксируются вовсе.
- 4.3 Согласованность означает сохранение заданных правил целостности. Например, завершённая транзакция не должна нарушать ограничение внешнего ключа.
- 4.4 Изоляция определяет взаимодействие одновременно выполняемых транзакций. Конкретные допустимые эффекты зависят от выбранного уровня изоляции.
- 4.5 Долговечность означает сохранение подтверждённых изменений после сбоя в пределах гарантий и настроек системы хранения. Вместе четыре свойства называют ACID.
- 4.6 В учебной операции создаются материал и запись его учёта размера. Если второе действие завершается ошибкой до COMMIT, ROLLBACK отменяет и создание материала.
- 4.7 Резервная копия нужна для восстановления данных. Сам факт создания копии недостаточен: её пригодность проверяют восстановлением в отдельное тестовое окружение.
- 4.8 RPO - максимально допустимый период потери данных. RTO - максимально допустимое время восстановления работы. Это разные цели, которые задают заранее.

ПРОВЕРОЧНЫЙ ПРИМЕР

Вымышленная учебная система задаёт RPO=15 минут и RTO=60 минут. Это означает допустимую потерю не более 15 минут изменений и целевое восстановление работы не дольше 60 минут. Эти числа являются условиями примера, а не обещанием реального сервиса.